

AWS Security , Monitoring, Scaling, and Billing

Study Guide

AMARJIT SEAL
AI&DS, PIET
Parul University

MODULE 1: Identity and Access Management (IAM)

1.1 Introduction to IAM

AWS Identity and Access Management (IAM) is a web service that allows you to securely control access to AWS services and resources for your users. IAM enables you to manage who is authenticated (signed in) and authorized (has permissions) to use resources.

💡 IAM is a GLOBAL service — it is not region-specific. Users, groups, roles, and policies apply across all AWS regions.

Key IAM Concepts

Concept	Description
IAM User	An entity representing a person or application that interacts with AWS. Has permanent credentials (username/password or access keys).
IAM Group	A collection of IAM users. Policies attached to a group apply to all its members. Users can belong to multiple groups.
IAM Role	An identity with permissions that can be assumed temporarily by users, applications, or AWS services. No long-term credentials.
IAM Policy	A JSON document defining permissions (Allow/Deny) for actions on resources. Attached to users, groups, or roles.
Root Account	Created when AWS account is first set up. Has full access. Should NEVER be used for daily tasks. Enable MFA immediately.
MFA	Multi-Factor Authentication — adds a second verification method beyond username/password for extra security.

1.2 IAM Users

- Represent individual people or services that interact with AWS
- Two types of credentials: Console access (password) and Programmatic access (Access Key ID + Secret Access Key)
- By default, a new IAM user has NO permissions — principle of least privilege
- Users can be assigned directly to policies or via groups
- Best practice: Do not share IAM user credentials
- Use access keys for CLI and SDK; rotate them regularly

IAM User Best Practices

- Delete root account access keys immediately after account creation
- Enable MFA on root and all privileged accounts
- Use IAM roles instead of sharing credentials
- Use groups to assign permissions — do not attach policies directly to individual users
- Apply least privilege — grant only the permissions required

1.3 IAM Groups

Groups are collections of IAM users. Groups make it easier to manage permissions for multiple users at once.

- Groups cannot be nested (no groups within groups)
- A user can belong to multiple groups (up to 10 groups per user)
- Policies attached to a group apply to ALL users in that group
- Groups are used to organize users by job function: Admins, Developers, Testers, etc.

💡 Example: Create a 'Developers' group with AmazonEC2FullAccess and AmazonS3ReadOnlyAccess policies. All developers added to this group automatically get those permissions.

1.4 IAM Roles

Roles are similar to users but are meant to be assumed temporarily. Roles do not have permanent credentials — they provide temporary security credentials when assumed.

When to Use IAM Roles

- Allow EC2 instances to access S3 without hardcoding credentials
- Cross-account access — allow another AWS account to access your resources
- Federated identity — allow users authenticated via Google, Facebook, or corporate directory to access AWS
- AWS service access — allow Lambda to write to DynamoDB, etc.

Role Type	Use Case
EC2 Instance Role	Grants EC2 permissions to access S3, DynamoDB, CloudWatch, etc.
Lambda Execution Role	Grants Lambda permissions to write logs to CloudWatch, access other services

Cross-Account Role	Allows users in Account A to assume a role in Account B
Service-Linked Role	Predefined by AWS services like Auto Scaling, ELB for automatic operations
Web Identity Federation	Allows mobile/web app users authenticated via Google/Facebook to access AWS

1.5 IAM Policies

Policies are JSON documents that define what actions are allowed or denied on which AWS resources. Policies are the fundamental mechanism for granting permissions in AWS.

Policy Structure (JSON)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3::my-bucket/*",
      "Effect": "Allow"
    },
    {
      "Action": "s3:DeleteObject",
      "Resource": "arn:aws:s3::my-bucket/*",
      "Effect": "Deny"
    }
  ]
}
```

Policy Elements

Element	Description
Version	Policy language version. Always use '2012-10-17'
Statement	Main element; one or more individual permission blocks
Effect	Either 'Allow' or 'Deny'. Deny always overrides Allow
Action	Specific API actions (e.g., 's3:GetObject', 'ec2:StartInstances')
Resource	ARN of the specific resource the statement applies to
Condition	Optional — restricts when the policy applies (e.g., specific IP, time)
Principal	Used in resource-based policies to specify who the policy applies to
NotAction	Allows all actions EXCEPT the ones listed
NotResource	Applies to all resources EXCEPT the ones listed

Types of IAM Policies

- **AWS Managed Policies: Predefined by AWS (e.g., AdministratorAccess, AmazonS3ReadOnlyAccess)**
 - Maintained by AWS, updated when new services are added
 - Can be attached to multiple identities
- **Customer Managed Policies: Created and managed by you**

- More granular and tailored to your needs
- Reusable across users, groups, roles
- **Inline Policies: Directly embedded in a single user, group, or role**
 - Deleted when the identity is deleted
 - Not reusable — not recommended for most use cases
- **Resource-Based Policies: Attached to a resource (e.g., S3 bucket policy, SQS queue policy)**
 - Specify who can access the resource

1.6 Policy Evaluation Logic

- By default, all requests are implicitly DENIED
- An explicit Allow in any applicable policy grants the permission
- An explicit Deny in ANY policy OVERRIDES any Allow — this is absolute
- AWS evaluates: Service Control Policies (SCPs) > Resource Policies > Identity Policies > Session Policies

 *Key Rule: Explicit DENY always wins. If any policy denies an action, the action is denied regardless of other allows.*

1.7 Implementing IAM Policies — Practical Steps

Step 1: Create an IAM Policy

1. Navigate to IAM Console > Policies > Create Policy
2. Choose service, actions, and resources using the visual editor OR JSON tab
3. Review and name the policy (e.g., 'S3ReadOnly-Production')
4. Create the policy

Step 2: Attach Policy to User/Group/Role

5. Go to IAM > Users/Groups/Roles
6. Select the identity, click 'Add Permissions'
7. Attach the policy directly or through a group

IAM Policy Conditions — Common Examples

Condition Key	Example Use
aws:SourceIp	Restrict access to specific IP ranges only
aws:RequestedRegion	Allow actions only in specific AWS regions
aws:MultiFactorAuthPresent	Require MFA before performing sensitive actions

s3:prefix

Restrict access to specific S3 folder prefixes

aws:CurrentTime

Allow access only during business hours

MODULE 2: AWS CloudWatch — Monitoring Resources

2.1 What is Amazon CloudWatch?

Amazon CloudWatch is a monitoring and observability service that collects data in the form of logs, metrics, and events from AWS resources and applications. It provides a unified view of operational health and enables automated responses to operational changes.

CloudWatch operates at the Infrastructure level. For application-level tracing, use AWS X-Ray. For API activity audit logs, use AWS CloudTrail.

2.2 CloudWatch Core Components

Component	Description
Metrics	Time-ordered data points about your resources (e.g., CPU utilization, disk I/O, network traffic)
Alarms	Watch a metric and trigger actions when threshold is breached (e.g., send SNS notification, trigger Auto Scaling)
Logs	Collect, monitor, and store log files from EC2, Lambda, CloudTrail, Route 53, and custom sources
Events (EventBridge)	Respond to state changes in AWS resources; schedule automated tasks using cron expressions
Dashboards	Custom visualizations — graphs, numbers, text widgets to monitor health at a glance
Insights	Query CloudWatch Logs with SQL-like language for analysis
Synthetics	Canary scripts that test endpoints and APIs periodically
ServiceLens	Unified monitoring combining traces, metrics, logs and alarms

2.3 CloudWatch Metrics

Default (Basic) Monitoring vs. Detailed Monitoring

Feature	Basic Monitoring Detailed Monitoring
Frequency	5-minute intervals 1-minute intervals
Cost	Free Additional charge

Available for

All EC2 instances by default | Must be explicitly enabled

Use case

Standard workloads | Applications requiring faster response

Key Default EC2 Metrics

- CPUUtilization — % of CPU being used
- NetworkIn / NetworkOut — bytes received/sent over all network interfaces
- DiskReadOps / DiskWriteOps — I/O operations to instance store volumes
- StatusCheckFailed — system and instance status check failures

💡 IMPORTANT: CloudWatch does NOT monitor RAM (memory) usage by default! You must install the CloudWatch Agent on the EC2 instance to collect memory metrics.

Custom Metrics

You can publish your own metrics using the AWS CLI or SDK. Custom metrics allow you to monitor application-specific data like:

- Memory utilization
- Disk usage percentage
- Request rates, error rates, response times
- Business KPIs (orders processed, active users)

2.4 CloudWatch Alarms

CloudWatch Alarms monitor a metric over a defined time period and perform actions when the metric crosses a threshold.

Alarm States

State	Meaning
OK	The metric is within the defined threshold
ALARM	The metric has breached the threshold
INSUFFICIENT_DATA	Not enough data to determine the state (usually on startup)

Alarm Actions

- Send notification via Amazon SNS (email, SMS, HTTP endpoint)
- Trigger Auto Scaling policies (scale out or scale in)

- Stop, terminate, reboot, or recover EC2 instances
- Trigger AWS Lambda functions

Creating an Alarm — Steps

8. CloudWatch Console > Alarms > Create Alarm
9. Select the metric (e.g., EC2 > Per-Instance Metrics > CPUUtilization)
10. Define threshold (e.g., CPU > 80% for 2 consecutive 5-minute periods)
11. Configure action (e.g., notify via SNS topic)
12. Name and create the alarm

2.5 CloudWatch Logs

CloudWatch Logs lets you monitor, store, and access log files from EC2 instances, Lambda functions, CloudTrail, and other sources.

Key Concepts

Term	Description
Log Group	Container for log streams with the same retention and access control settings
Log Stream	Sequence of log events from a single source (e.g., one EC2 instance)
Log Event	A single record of activity with a timestamp and message
Log Retention	Configurable from 1 day to 10 years; never expire by default
Metric Filter	Pattern matching on logs to extract metrics (e.g., count ERROR occurrences)
Subscription Filter	Stream log data in real-time to Kinesis, Lambda, or Elasticsearch

2.6 CloudWatch vs CloudTrail vs AWS Config

Service	Primary Purpose
CloudWatch	Monitor performance metrics, set alarms, collect logs, automate responses
CloudTrail	Audit trail of ALL API calls made to AWS (who did what, when, from where)
AWS Config	Track configuration changes of AWS resources over time for compliance

MODULE 3: Elastic Load Balancing (ELB)

3.1 What is Elastic Load Balancing?

Elastic Load Balancing (ELB) automatically distributes incoming application traffic across multiple targets — such as EC2 instances, containers, IP addresses, and Lambda functions — in one or more Availability Zones. It ensures high availability and fault tolerance.

Why Use Load Balancing?

- Distribute traffic evenly across multiple servers to prevent overloading any single server
- Automatic health checks — route traffic only to healthy targets
- Increase availability — if one instance fails, traffic is routed to healthy instances
- Enable horizontal scaling — add or remove instances without downtime
- SSL/TLS termination — offload encryption/decryption from backend instances

3.2 Types of AWS Load Balancers

Type	Best Used For
Application Load Balancer (ALB)	HTTP/HTTPS traffic; Layer 7; path-based and host-based routing; ideal for microservices and containers
Network Load Balancer (NLB)	TCP/UDP/TLS traffic; Layer 4; ultra-high performance; millions of requests/sec; low latency; static IP
Gateway Load Balancer (GWLB)	Deploy, scale, and manage third-party network virtual appliances (firewalls, IDS/IPS); Layer 3
Classic Load Balancer (CLB)	Legacy; Layer 4 and 7; not recommended for new applications; being deprecated

3.3 Application Load Balancer (ALB) — Deep Dive

Key Features

- Operates at Layer 7 (Application layer) — understands HTTP/HTTPS
- Content-based routing: route based on URL path (/images, /api), hostname, HTTP headers, query strings
- Supports WebSockets and HTTP/2
- Target types: EC2 instances, ECS containers, Lambda functions, IP addresses
- Sticky sessions (session affinity) using cookies
- Integration with AWS WAF (Web Application Firewall)

- Native support for containerized applications

ALB Routing Rules — Examples

Rule Type	Example
Path-based	example.com/api/* → API servers; example.com/images/* → Image servers
Host-based	api.example.com → API service; www.example.com → Web service
HTTP Header	Route based on User-Agent (mobile vs. desktop)
Query String	example.com?version=2 → v2 backend
Source IP	Route internal traffic to internal services

3.4 Network Load Balancer (NLB)

- Layer 4 — works with TCP, UDP, TLS protocols
- Handles millions of requests per second with ultra-low latency (< 1ms)
- Preserves the source IP address of the client
- Provides static IP addresses and supports Elastic IPs
- Best for gaming, IoT, financial trading platforms

3.5 Load Balancer Components

Component	Description
Listener	Checks for connection requests; configured with protocol and port (e.g., HTTP:80, HTTPS:443)
Target Group	Group of registered targets (EC2 instances, IPs, Lambda); each group has its own health check config
Health Check	Periodically tests targets; unhealthy targets are removed from rotation
Security Group	Controls inbound/outbound traffic to the load balancer
Access Logs	Captured and stored in S3; useful for auditing and troubleshooting
Sticky Sessions	Ensure the same client always reaches the same target (useful for session state)

3.6 ELB Health Checks

Health checks ensure traffic is only sent to healthy targets. The load balancer periodically sends requests to targets and marks them healthy or unhealthy based on the response.

- HealthyThresholdCount: consecutive successful checks before marking healthy (default: 3)
- UnhealthyThresholdCount: consecutive failed checks before marking unhealthy (default: 3)
- HealthCheckIntervalSeconds: time between health checks (default: 30s)
- HealthCheckTimeoutSeconds: time to wait for a response (default: 5s)
- HTTP health checks look for 200 OK response by default

MODULE 4: Amazon EC2 Auto Scaling

4.1 What is Auto Scaling?

Amazon EC2 Auto Scaling automatically adjusts the number of EC2 instances in response to changing demand. It helps you maintain application availability and enables you to scale your EC2 capacity up or down automatically according to conditions you define.

Benefits of Auto Scaling

- High Availability: Replace unhealthy instances automatically
- Cost Optimization: Run only the instances you need, reducing costs during low demand
- Fault Tolerance: Distribute instances across multiple AZs
- Elasticity: Scale out (add instances) during peaks, scale in (remove instances) during off-peak

4.2 Auto Scaling Components

Component	Description
Launch Template / Launch Configuration	Blueprint for new EC2 instances: AMI ID, instance type, key pair, security groups, storage
Auto Scaling Group (ASG)	Logical grouping of EC2 instances. Defines min, max, and desired capacity
Scaling Policies	Define when and how to scale (target tracking, step, simple, scheduled)
Minimum Capacity	Minimum number of instances always running (e.g., 2)
Maximum Capacity	Maximum number of instances allowed (e.g., 10)
Desired Capacity	Current target number of instances (e.g., 4). Auto Scaling maintains this
Cooldown Period	Time (default 300s) after a scaling activity during which no further scaling happens

4.3 Scaling Policies

1. Target Tracking Scaling

Most common and recommended. Maintains a target value for a specific metric automatically.

- Example: Keep average CPU utilization at 50%
- Auto Scaling adds/removes instances to keep the metric near the target

- Similar to a thermostat — set the desired temperature, system maintains it

2. Step Scaling

Scales in steps based on how much the metric has breached the threshold.

- Example: CPU 70-80% → add 1 instance; CPU 80-90% → add 2 instances; CPU > 90% → add 3 instances
- Provides more granular control than simple scaling

3. Simple Scaling

Single scaling action when a CloudWatch alarm is triggered. Waits for cooldown period before next action.

- Add 2 instances when CPU > 80%; Remove 1 instance when CPU < 30%

4. Scheduled Scaling

Scale based on predictable, known traffic patterns.

- Example: Scale to 10 instances every Monday at 8 AM; scale back to 2 at 6 PM
- Useful for recurring events, batch jobs, business-hours traffic

5. Predictive Scaling

Uses machine learning to analyze historical patterns and forecast future demand.

- Proactively scales before demand arrives
- Combines forecast and actual demand

4.4 Auto Scaling with ELB

Auto Scaling and Elastic Load Balancing work together to create a highly available and scalable architecture:

- Register the Auto Scaling Group with a Target Group on the load balancer
- New instances are automatically registered with the ELB when launched
- Instances are deregistered from ELB before termination (connection draining)
- ELB health checks can replace Auto Scaling health checks
- The ELB distributes traffic only to healthy, running instances

💡 Architecture Pattern: Internet → ELB → Auto Scaling Group (EC2 instances across multiple AZs) → Database (Multi-AZ RDS). This is the standard highly-available, scalable web application architecture.

4.5 Auto Scaling Termination Policies

Policy	Description
Default	Balance AZs first, then terminate instance closest to next billing hour
OldestInstance	Terminate oldest running instance (helps update to new instance types)
NewestInstance	Terminate newest instance (useful for testing)
OldestLaunchTemplate	Terminate instances with oldest launch template/configuration
ClosestToNextInstanceHour	Minimize billing costs by terminating instance closest to next hour
AllocationStrategy	Optimize instance mix for Spot/On-Demand

MODULE 5: Cloud Economics and Billing

5.1 AWS Pricing Models

Pricing Model	Description
Pay-as-you-go	Pay only for what you use, when you use it. No upfront costs, no termination fees.
Save when you reserve	Reserve capacity for 1 or 3 years; save up to 75% vs. On-Demand.
Pay less by using more	Volume discounts — the more you use, the lower the per-unit cost (S3, data transfer).
Pay less as AWS grows	AWS passes economies of scale savings to customers through price reductions.
Free Tier	12 months free on 100+ services; always-free tiers for select services.

5.2 EC2 Purchasing Options

Option	Description & Use Case
On-Demand	Pay by the hour/second; no commitment; highest cost; ideal for unpredictable workloads
Reserved Instances (RIs)	1 or 3 year commitment; up to 72% savings; ideal for steady-state workloads
Spot Instances	Bid on unused EC2 capacity; up to 90% savings; can be interrupted with 2-min notice
Savings Plans	Flexible pricing model (up to 66% savings); commit to \$/hour spend for 1-3 years
Dedicated Hosts	Physical server dedicated to you; helps with licensing compliance (BYOL)
Dedicated Instances	Instances run on hardware dedicated to one customer; no control over host placement
Capacity Reservations	Reserve capacity in a specific AZ for any duration; pay On-Demand rate

Reserved Instance Types

RI Type	Description
---------	-------------

Standard RI	Up to 72% discount; cannot change instance type; highest savings
Convertible RI	Up to 54% discount; can exchange for different instance type, OS, tenancy
Scheduled RI	Reserve for specific time windows (e.g., every Friday 6-10 PM)

5.3 Total Cost of Ownership (TCO)

TCO compares the total cost of running infrastructure on-premises versus on AWS. AWS TCO benefits include:

- Elimination of hardware purchase, data center space, power, cooling costs
- Reduced staffing for infrastructure maintenance
- No over-provisioning — pay for actual usage
- Economies of scale — AWS passes savings to customers
- Capital expenditure (CapEx) → Operational expenditure (OpEx) conversion

5.4 AWS Billing Tools

Tool	Purpose
AWS Cost Explorer	Visualize, understand, and manage costs over time; forecast future spend; filter by service, tag, account
AWS Budgets	Set custom cost and usage budgets; receive alerts when thresholds are exceeded
Cost and Usage Report	Most detailed billing data; line-item costs; integrates with Athena and QuickSight
AWS Pricing Calculator	Estimate costs before deployment; build architecture scenarios
Cost Allocation Tags	Tag resources to track costs by project, department, or environment
Billing Dashboard	Overview of current month spend, forecasts, and service breakdown
AWS Organizations	Consolidated billing for multiple AWS accounts; volume discounts applied across org

5.5 AWS Support Plans

Plan	Features
Basic (Free)	Documentation, whitepapers, forums, AWS Trusted Advisor (7 checks), Personal Health Dashboard

Developer (\$29/mo)	Business hours access, 1 contact, general guidance < 24 hrs, system impaired < 12 hrs
Business (\$100/mo)	24/7 support, unlimited contacts, all Trusted Advisor checks, < 1 hr critical response
Enterprise On-Ramp (\$5,500/mo)	Pool of Technical Account Managers, < 30 min critical response
Enterprise (\$15,000/mo)	Dedicated TAM, < 15 min critical response, operational reviews, training

5.6 AWS Free Tier

- Always Free: Never expire — Lambda (1M requests/month), DynamoDB (25 GB storage), etc.
- 12 Months Free: From account creation — EC2 (750 hrs t2.micro/month), S3 (5 GB), RDS (750 hrs), etc.
- Trials: Short-term free trials for select services

5.7 Cost Optimization Strategies

- Right-sizing: Match instance type to workload requirements using CloudWatch metrics
- Reserved Instances/Savings Plans for predictable workloads
- Spot Instances for fault-tolerant, flexible workloads
- Auto Scaling to run only needed instances
- S3 Intelligent-Tiering to automatically move infrequently accessed data
- Delete unattached EBS volumes, unused Elastic IPs, and idle load balancers
- Use CloudFront to reduce data transfer costs from S3 and EC2
- Enable AWS Trusted Advisor recommendations for cost optimization

MODULE 6: Scale and Load Balance Your Architecture

6.1 Well-Architected Framework — Reliability Pillar

Scaling and load balancing are core to the Reliability and Performance Efficiency pillars of the AWS Well-Architected Framework.

Design Principles for Scalable Architecture

- Design for failure — assume components will fail; build redundancy
- Decouple components — use SQS, SNS to reduce dependencies
- Scale horizontally — add more small servers rather than larger ones
- Stop guessing capacity — use Auto Scaling instead of fixed provisioning
- Distribute across multiple Availability Zones

6.2 Highly Available, Scalable Web Application Architecture

A standard reference architecture for a scalable web application on AWS:

13. Route 53 (DNS) → resolves domain to ELB endpoint
14. Application Load Balancer → distributes HTTP/HTTPS traffic across instances
15. Auto Scaling Group → EC2 web servers in 2+ Availability Zones
16. Amazon RDS Multi-AZ → database with automatic failover
17. ElastiCache → reduce database load with in-memory caching
18. S3 + CloudFront → serve static assets globally with low latency
19. CloudWatch → monitor performance and trigger Auto Scaling alarms

6.3 Lab: Scale and Load Balance Your Architecture

The AWS Academy lab 'Scale and Load Balance Your Architecture' covers:

- Creating a Launch Template with a web server AMI
- Creating an Auto Scaling Group using the Launch Template
- Configuring an Application Load Balancer and Target Group
- Attaching the Auto Scaling Group to the Target Group
- Creating CloudWatch Alarms to trigger scaling policies
- Testing auto scaling by generating load (stress testing)
- Monitoring the ELB and Auto Scaling Group in CloudWatch

 *Exam Tip: Know the correct order — Launch Template → Auto Scaling Group → Target Group → Load Balancer → Listener → Routing Rules. Alarms trigger scaling policies which add/remove instances to the ASG.*

SECTION 7: Important University Exam Questions

The following questions are compiled from university exam patterns, AWS Academy knowledge checks, Cloud Practitioner and Solutions Architect certification exams, and common academic assessment styles.

7.1 Short Answer / Definition Questions (2-4 Marks)

Q1 Define IAM. What are its four main components? (IAM Users, Groups, Roles, Policies)

Q2 What is the difference between an IAM User and an IAM Role?

Q3 What is the principle of least privilege? How does IAM enforce it?

Q4 What is an IAM Policy? Describe the four key elements of a policy JSON document.

Q5 What is Amazon CloudWatch? List any four services it monitors.

Q6 What is the difference between CloudWatch, CloudTrail, and AWS Config?

Q7 Define Elastic Load Balancing. What problem does it solve?

Q8 What is the difference between an Application Load Balancer and a Network Load Balancer?

Q9 What is EC2 Auto Scaling? What are the three capacity parameters it uses?

Q10 What is the AWS Free Tier? Name any three always-free services.

7.2 Long Answer / Descriptive Questions (8-15 Marks)

Q11

Explain the structure and components of an IAM Policy. Write a sample IAM Policy JSON that allows a user to read from an S3 bucket named 'company-data' but denies the ability to delete objects. Explain each element.

Q12

Describe the process of implementing IAM best practices in an AWS organization. Include: root account security, MFA, groups, roles, and the principle of least privilege.

Q13

Explain how Amazon CloudWatch monitors EC2 instances. What metrics are collected by default? What additional steps are needed to monitor memory utilization? How do CloudWatch Alarms trigger Auto Scaling?

Q14

Compare the three types of AWS Load Balancers (ALB, NLB, CLB). With the help of a diagram or architecture description, explain how an Application Load Balancer routes HTTP requests to different backend services based on URL paths.

Q15

A web application on AWS needs to handle unpredictable traffic spikes. Design a scalable, highly available architecture using Auto Scaling, Elastic Load Balancing, and CloudWatch. Describe each component's role and how they interact.

Q16

Explain the five EC2 pricing models: On-Demand, Reserved, Spot, Savings Plans, and Dedicated Hosts. For each, describe the use case, cost savings, and commitment required. Which would you recommend for a 24/7 production database? Why?

Q17

What is Total Cost of Ownership (TCO)? How does migrating to AWS reduce TCO? Discuss at least five cost factors that are eliminated or reduced when moving from on-premises to AWS.

Q18

Describe the four types of Auto Scaling policies: Target Tracking, Step Scaling, Simple Scaling, and Scheduled Scaling. For each, provide an example scenario where it would be the most appropriate choice.

7.3 Multiple Choice / MCQ-Style Questions

Q19

Which IAM feature is used to grant temporary access to AWS resources to applications running on EC2 instances? a) IAM Users with access keys b) IAM Groups with permissions c) IAM Roles attached to EC2 instances ✓ d) IAM Policies with condition keys

Q20

A company wants to allow users from its corporate Active Directory to access AWS without creating separate IAM users. Which IAM feature enables this? a) IAM Groups b) IAM Policies c) Identity Federation ✓ d) IAM Access Keys

Q21

What happens when both an Allow and an explicit Deny exist in AWS IAM policies for the same action? a) Allow takes precedence b) Deny takes precedence ✓ c) The most recently attached policy wins d) Requires administrator approval

Q22

Which CloudWatch metric is NOT available by default for EC2 instances and requires the CloudWatch Agent? a) CPUUtilization b) NetworkIn c) MemoryUtilization ✓ d) DiskReadOps

Q23

Which type of AWS Load Balancer operates at OSI Layer 7 and supports content-based routing? a) Network Load Balancer b) Classic Load Balancer c) Gateway Load Balancer d) Application Load Balancer ✓

Q24

An e-commerce company expects heavy traffic every December. Which Auto Scaling policy should they use to proactively scale up resources before December 1st? a) Target Tracking Scaling b) Step Scaling c) Scheduled Scaling ✓ d) Simple Scaling

Q25

Which EC2 purchasing option offers up to 90% cost savings but can be interrupted with a 2-minute warning? a) Reserved Instances b) Dedicated Hosts c) On-Demand Instances d) Spot Instances ✓

Q26

A startup wants to monitor AWS costs and receive an email alert when monthly spending exceeds \$500. Which combination of services should they use? a) CloudWatch + SNS b) AWS Budgets + SNS ✓ c) Cost Explorer + Lambda d) CloudTrail + SES

Q27

Which component of an Auto Scaling Group defines the AMI, instance type, and security groups for new instances? a) Scaling Policy b) Health Check c) Launch Template ✓ d) Target Group

Q28

What is the purpose of a 'Cooldown Period' in Auto Scaling? a) Time to shut down instances gracefully b) Time to wait before the next scaling activity ✓ c) Time for health checks to complete d) Time for ELB to register new instances

7.4 Case Study / Scenario-Based Questions (15-20 Marks)

Q29

CASE STUDY: A fintech startup is launching a payment processing application on AWS. The application must be: • Highly available (99.99% uptime SLA) • Secure (PCI-DSS compliant) • Scalable (handles 100 to 10,000 transactions/minute) • Cost-optimized Design the complete AWS architecture. Specify: IAM roles and policies, load balancing strategy, auto scaling configuration, monitoring approach, and cost optimization measures.

Q30

CASE STUDY: A company's DevOps team needs to set up monitoring and alerting for their production environment. They want to: • Alert the ops team when CPU > 80% for 5 minutes • Automatically scale out 2 EC2 instances when CPU > 70% • Alert when 5XX error rate > 1% on their Application Load Balancer • Send daily cost reports when spend exceeds \$1,000/day Describe how to configure CloudWatch, Auto Scaling policies, and AWS Budgets to meet these requirements. Include alarm states, actions, and escalation paths.

Q31

CASE STUDY: An online video streaming platform currently runs on-premises with 50 physical servers. They are planning to migrate to AWS. Compare the Total Cost of Ownership (TCO) for both options, considering hardware costs, staffing, power, cooling, redundancy, and scaling. What AWS services and pricing models would you recommend to optimize costs?

7.5 True/False & Fill in the Blanks

Q32

TRUE or FALSE: a) IAM is a regional service — each region has its own IAM. [FALSE — IAM is Global] b) An IAM Role can be assumed by an EC2 instance. [TRUE] c) CloudWatch monitors RAM by default on EC2. [FALSE — needs CloudWatch Agent] d) An explicit Deny in IAM can be overridden by an Allow. [FALSE — Deny always wins] e) Spot Instances can be used for databases requiring persistent state. [FALSE — not recommended] f) A Network Load Balancer operates at OSI Layer 7. [FALSE — Layer 4]

Q33

FILL IN THE BLANKS: a) IAM stands for _____ and _____ Management. b)

The _____ pricing model requires 1 or 3 year commitment for up to 72% savings. c) CloudWatch collects data in three forms: _____, _____, and events. d) The maximum number of instances in an Auto Scaling Group is controlled by _____ capacity. e) ELB performs _____ checks to ensure traffic only goes to healthy instances. f) AWS _____ provides consolidated billing for multiple AWS accounts.

SECTION 8: Quick Revision Cheat Sheet

IAM Quick Reference

Key Point	Detail
Root Account	Full access; secure with MFA; never use for daily tasks; delete access keys
Policy Priority	Explicit Deny > Explicit Allow > Implicit Deny
Policy Types	AWS Managed, Customer Managed, Inline, Resource-Based
Role vs User	User = permanent credentials; Role = temporary, assumed, no long-term credentials
IAM Scope	Global — not regional
Max Groups per User	10
Groups in Groups	NOT ALLOWED

CloudWatch Quick Reference

Key Point	Detail
Default EC2 Monitoring	Every 5 minutes (free)
Detailed Monitoring	Every 1 minute (paid)
RAM Monitoring	NOT default — install CloudWatch Agent
Alarm States	OK, ALARM, INSUFFICIENT_DATA
Alarm Actions	SNS notification, Auto Scaling, EC2 actions, Lambda
CloudWatch vs CloudTrail	CloudWatch = performance; CloudTrail = API audit trail

ELB Quick Reference

Key Point	Detail
ALB	Layer 7, HTTP/HTTPS, path/host-based routing, WAF integration
NLB	Layer 4, TCP/UDP, millions RPS, ultra-low latency, static IP
GWLB	Layer 3, for virtual network appliances (firewalls)
CLB	Legacy, Layer 4+7, being deprecated

Health Checks

Automatically removes unhealthy targets from rotation

Sticky Sessions

Same client → same target (using cookies)

Auto Scaling Quick Reference

Key Point	Detail
Cooldown Period	300 seconds (default) — no scaling during cooldown
Health Checks	EC2 (default) or ELB health checks
Min/Max/Desired	Min = always running; Max = upper limit; Desired = target count
Target Tracking	Most recommended — maintains target metric automatically
Scheduled Scaling	For predictable patterns (time-based)
Termination Default	Balance AZs first, then closest to billing hour

Billing & Pricing Quick Reference

Key Point	Detail
On-Demand	Highest cost; no commitment; flexible
Reserved (1yr)	Up to 40% savings; Standard or Convertible
Reserved (3yr)	Up to 72% savings
Spot	Up to 90% savings; can be interrupted 2 minutes' notice
Savings Plans	Up to 66%; flexible — commit to \$/hr not specific instance
AWS Budgets	Set alerts for cost/usage thresholds
Cost Explorer	Analyze and forecast costs; historical and future
Free Tier	12 months (some services); Always Free (Lambda, DynamoDB)

— End of Document —

Best of luck in your examinations!